Summary	Part 1: Create a Streamlit interface to execute 5 TPC-DS queries using Snowflake SQLAlchemy, implementing Qualification Substitution Parameters as variables, and validating input values. Part 2: Build a CLV model using the TPC-DS dataset, host it, and develop a Streamlit interface for users to input features, invoke the model, and display the CLV predictions.
URL	https://ne-tpcds-customer-lifetime-valuecustomer-lifetime-value-au9kfr.streamlit.app/
Category	Web
Environment	SnowPark, Jupyter, Streamlit
Status	Published
Author	Mohit Negi Prithvi Vasireddy

Introduction

Part-1: Validating and running Queries Via SnowFlake Alchemy in Python

Getting set up

Implementation

Defining Queries and indexing them

Create Engine() and retreive creds from config file

Customer Lifecycle Value Model and Streamlit interface

Part-2: Training and Modeling an XGB model for CLV prediction

Getting Started

Create databases, warehouses, schemas and configure them as relevant

Feature Engineering

Congratulations

Introduction

Here, we are building a streamlit interface that connects to the Snowflake database, executes TPC-DS queries with Qualification Substitution Parameters as variables, validates user inputs, and provides personalized marketing recommendations based on the estimated lifetime value of each customer.

Last Updated: 2023-03-17

Part-1 – Validating and Running TPCDS Queries via Snowflake SQLAlchemy in Python:

In this part, we build a user-friendly interface for executing TPC-DS queries using Snowflake SQLAlchemy in Python. This interface allows users to interact with the underlying database without needing to write complex SQL queries or understand the intricacies of the database schema. The main goal is to make it easy for users to input necessary values and retrieve the desired results, simplifying the process of data exploration and analysis.

To achieve this, we use Streamlit, a popular open-source framework for creating web applications in Python, as our front-end interface. Streamlit allows us to quickly build interactive and dynamic web applications with minimal effort, making it an ideal choice for this project.

In the back-end, we use Snowflake SQLAlchemy, a powerful and versatile library for connecting to and querying Snowflake databases in Python. By leveraging SQLAlchemy, we can interact with Snowflake seamlessly and execute complex TPC-DS queries with ease.

Throughout the project, we follow the TPC-DS documentation to ensure that we correctly implement the Qualification Substitution Parameters as variables. We also validate user inputs, such as date fields and other specific constraints, to maintain the integrity of the data and the results we generate.

Getting set up

Bash, SnowFlake

Installing the required libraries and setting up the snowflake environment with the relevant user/account credentials and warehouse settings as suggested for the Snowflake TPCDS dataset.

1. Create a conda environment using the provided *environment.yml* file.
- conda env create -f environment.yml
- Activate that created conda environment by conda activate snowpark_ml_test
2. Edit the *creds.json* file to with your account information to connect to your account.
3. Load Jupyter or equivalent notebook to begin executing the notebook.

Implementation

Jupyter

/* Import required libraries */

```
import pandas as pd
import numpy as np
from snowflake.sqlalchemy import URL
from sqlalchemy import create_engine
import streamlit as st
import sqlalchemy
from streamlit_lottie import st_lottie
import requests
import plotly.express as px
```

accuracy_calc() and run_query().

**Generate SQL queries based on the input parameters provided to the run_query() function.* These queries are used to retrieve data on customers' predictions and actual sales, taking into consideration the customers' demographic and financial information.

accuracy_calc(df) function:
Takes a DataFrame df as input, which is expected to have columns 'predicted_value' and 'actual_sales'. Returns the accuracy by calculating the absolute difference between the predicted and actual sales values, divided by the actual sales, and then subtracting this value from 1.

run_query() function:
Takes the following input parameters: dob_list, education_option, gender_option, dept_option, credit_option, and marital_option. These parameters represent the filter criteria for the query.

The function initializes two base SQL queries, query_prime1 and query_prime12, both of which retrieve the sum of predictions and actual sales from a table called 'predictions'.
Then, the function sets up a series of conditionals to check which filter criteria are present in the input parameters.

If a filter criteria has only one value, the corresponding position in the str_pos list is set to -1, and the query will use an exact match for that filter. If there are multiple values for a filter, the corresponding position in str_pos is set to 1, and the query will use an IN clause for the filter. If no values are provided for a filter, the corresponding position in str_pos is set to 0, and the filter is ignored.

Based on the values in the str_pos list, the function appends the relevant filter conditions to the base SQL queries query_prime1 and query_prime12.
For the query_prime12 query, it finally appends a GROUP BY and ORDER BY clause to group the results by the customers' birth year.

The function returns both generated queries: query_prime1 and query_prime12.

Hence, we generate queries based on input parameters to be filtered and retrieve data about customers' predictions and actual sales while considering their demographic and financial information. It also defines an accuracy calculation function for evaluating the predictions.

```
# Define the accuracy_calc function that takes a DataFrame as input and returns the accuracy value

def accuracy_calc(df):
    return df['predicted_value'] / (df['predicted_value'] + df['actual_sales'])

# Define the run_query function that takes several input parameters and returns two SQL query strings

def run_query(dob_list, education_option, gender_option, dept_option, credit_option, marital_option):
    # Initialize query_prime1 and query_prime12 with their base SQL query strings
    query_prime1 = """select sum(prediction) Predicted, sum(actual_sales) Actual_Sales
                    from predictions
                    where c_birth_year between """ + str(dob_list[0]) + " and " + str(dob_list[1])

    query_prime12 = """select c_birth_year, sum(prediction) Predicted_value, sum(actual_sales) Actual_Sales
                     from predictions
                     where c_birth_year between """ + str(dob_list[0]) + " and " + str(dob_list[1])

    # Initialize an array of flags for query conditions

    str_pos = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

    # Update str_pos and gender_option based on the length of gender_option

    if len(gender_option) == 1:
        str_pos[5] = -1
        str_pos[0] = 0
    elif len(gender_option) >= 1:
        str_pos[5] = 0
        str_pos[0] = 1
    else:
        gender_option = gender_option + (1,)
        str_pos[5] = 0
        str_pos[0] = 0

    # Update str_pos and marital_option based on the length of marital_option

    if len(marital_option) == 1:
        str_pos[6] = -1
        str_pos[1] = 0
    elif len(marital_option) >= 1:
        str_pos[6] = 0
        str_pos[1] = 1
    else:
        marital_option = marital_option + (1,)
        str_pos[6] = 0
        str_pos[1] = 0

    # Update str_pos and dept_option based on the length of dept_option

    if len(dept_option) == 1:
        str_pos[7] = -1
```

```
    str_pos[2] = 0
elif len(dept_option) >= 1:
    str_pos[7] = 0
    str_pos[2] = 1
else:
    dept_option = dept_option + (1,)
    str_pos[7] = 0
    str_pos[2] = 0

# Update str_pos and credit_option based on the length of credit_option

if len(credit_option) == 1:
    str_pos[8] = -1
    str_pos[3] = 0
elif len(credit_option) >= 1:
    str_pos[8] = 0
    str_pos[3] = 1
else:
    credit_option = credit_option + (1,)
    str_pos[8] = 0
    str_pos[3] = 0

# Update str_pos and education_option based on the length of education_option

if len(education_option) == 1:
    str_pos[9] = -1
    str_pos[4] = 0
elif len(education_option) >= 1:
    str_pos[9]
```

The main() function here sets up two containers for different sections of the Streamlit app. The first container displays "Part - 1" and calls the st_part1 function, while the second container displays "Part - 2: Customer Lifetime Valuation using XGBoost" and calls the st_part2 function.

```
# Define a function to load a Lottie animation from a URL
def load_lottieurl(url: str):
    r = requests.get(url)
    if r.status_code != 200:
        return None
    return r.json()
```

```
# Define the main function for the Streamlit app
def main():
    # Create two containers for different sections of the app
    with st.container():
        st.header("Part - 1")
        st_part1()
    with st.container():
        st.header("Part - 2: Customer Lifetime Valuation using XGBoost")
        st_part2()

# Define a function for the first part of the Streamlit app
def st_part1():
    # Create two columns for layout within the container
    col1, col2 = st.columns(2, gap='small')
    with col1:
        # Load the Lottie animation from a URL
        lottie_url_hello =
"https://raw.githubusercontent.com/Negi97Mohit/snowpark-python-demos/main/tpcds-customer-lifetime
-value/97474-data-center.json"
        lottie_hello = load_lottieurl(lottie_url_hello)

        # Display the Lottie animation
        st_lottie(lottie_hello,
                reverse=True,
                height=400,
                width=400,
                speed=1,
                loop=True,
                quality='high',
                key='Car'
                )

        # Display the code for generating the query table within an expander
        with st.expander("Query table generator code"):
            p1code = """import pandas as pd
from snowflake.sqlalchemy import URL
from sqlalchemy import create_engine
```

Defining Queries (and indexing them)

We first define a list called queries containing five SQL queries in the form of multiline strings. Each SQL query performs a different analytical task on various tables such as customer,

store_sales, web_sales, catalog_sales, item, date_dim, etc. Here's a brief description of each query:

Query11.sql: This query retrieves information about customers who have a higher growth in online sales compared to in-store sales within the given date range. The result is ordered by customer_id, customer_first_name, customer_last_name, and customer_preferred_cust_flag and is limited to 100 rows.

Query12.sql: This query calculates the revenue and revenue ratio for items in the 'Sports', 'Books', and 'Home' categories sold within a specific date range. The result is ordered by i_category, i_class, i_item_id, i_item_desc, and revenueratio, and is limited to 100 rows.

Query13.sql: This query calculates the average sales quantities, average sales prices, average wholesale costs, and the total sum of wholesale costs for various customer demographics, store locations, and product price ranges. It also applies specific filters on marital status, education status, and dependent count of the customers.

Query14.sql: This query finds the total sales and number of sales for items sold through three different channels (store, catalog, and web) within a specific date range. The results are aggregated by brand, class, and category, and they are filtered to include only those rows where the sales are greater than the average sales calculated in a subquery. The final result is grouped by rollup (channel, i_brand_id, i_class_id, i_category_id) and is limited to 100 rows.

Query15.sql: This query calculates the total sales price for catalog sales within a given date range, grouped by the zip code of the customer's address. It filters the results based on specific zip codes, states, or sales prices. The result is ordered by ca_zip and limited to 100 rows.

```
queries=[
'''
with year_total as (
select c_customer_id customer_id
      ,c_first_name customer_first_name
      ,c_last_name customer_last_name
      ,c_preferred_cust_flag customer_preferred_cust_flag
      ,c_birth_country customer_birth_country
      ,c_login customer_login
      ,c_email_address customer_email_address
      ,d_year dyear
      ,sum(ss_ext_list_price-ss_ext_discount_amt) year_total
```

```
      ,'s' sale_type
from customer
    ,store_sales
    ,date_dim
where c_customer_sk = ss_customer_sk
  and ss_sold_date_sk = d_date_sk
group by c_customer_id
        ,c_first_name
        ,c_last_name
        ,c_preferred_cust_flag
        ,c_birth_country
        ,c_login
        ,c_email_address
        ,d_year
union all
select c_customer_id customer_id
      ,c_first_name customer_first_name
      ,c_last_name customer_last_name
      ,c_preferred_cust_flag customer_preferred_cust_flag
      ,c_birth_country customer_birth_country
      ,c_login customer_login
      ,c_email_address customer_email_address
      ,d_year dyear
      ,sum(ws_ext_list_price-ws_ext_discount_amt) year_total
      ,'w' sale_type
from customer
    ,web_sales
    ,date_dim
where c_customer_sk = ws_bill_customer_sk
  and ws_sold_date_sk = d_date_sk
group by c_customer_id
        ,c_first_name
        ,c_last_name
        ,c_preferred_cust_flag
        ,c_birth_country
        ,c_login
        ,c_email_address
        ,d_year
        )
  select
                  t_s_secyear.customer_id
                ,t_s_secyear.customer_first_name
                ,t_s_secyear.customer_last_name
                ,t_s_secyear.customer_preferred_cust_flag
from year_total t_s_firstyear
    ,year_total t_s_secyear
    ,year_total t_w_firstyear
    ,year_total t_w_secyear
```

```
where t_s_secyear.customer_id = t_s_firstyear.customer_id
        and t_s_firstyear.customer_id = t_w_secyear.customer_id
        and t_s_firstyear.customer_id = t_w_firstyear.customer_id
        and t_s_firstyear.sale_type = 's'
        and t_w_firstyear.sale_type = 'w'
        and t_s_secyear.sale_type = 's'
        and t_w_secyear.sale_type = 'w'
        and t_s_firstyear.dyear = 2001
        and t_s_secyear.dyear = 2001+1
        and t_w_firstyear.dyear = 2001
        and t_w_secyear.dyear = 2001+1
        and t_s_firstyear.year_total > 0
        and t_w_firstyear.year_total > 0
        and case when t_w_firstyear.year_total > 0 then t_w_secyear.year_total
/ t_w_firstyear.year_total else 0.0 end
            > case when t_s_firstyear.year_total > 0 then
t_s_secyear.year_total / t_s_firstyear.year_total else 0.0 end
order by t_s_secyear.customer_id
        ,t_s_secyear.customer_first_name
        ,t_s_secyear.customer_last_name
        ,t_s_secyear.customer_preferred_cust_flag
limit 100''',
'''select  i_item_id
      ,i_item_desc
      ,i_category
      ,i_class
      ,i_current_price
      ,sum(ws_ext_sales_price) as itemrevenue
      ,sum(ws_ext_sales_price)*100/sum(sum(ws_ext_sales_price)) over
          (partition by i_class) as revenueratio
from
  web_sales
      ,item
      ,date_dim
where
  ws_item_sk = i_item_sk
    and i_category in ('Sports', 'Books', 'Home')
    and ws_sold_date_sk = d_date_sk
  and d_date between cast('1999-02-22' as date)
                                 and dateadd(day,30,cast('1999-02-22' as date)
)
group by
  i_item_id
        ,i_item_desc
        ,i_category
        ,i_class
        ,i_current_price
order by
```

```
  i_category
        ,i_class
        ,i_item_id
        ,i_item_desc
        ,revenueratio
limit 100''',
''' select avg(ss_quantity)
      ,avg(ss_ext_sales_price)
      ,avg(ss_ext_wholesale_cost)
      ,sum(ss_ext_wholesale_cost)
from store_sales
    ,store
    ,customer_demographics
    ,household_demographics
    ,customer_address
    ,date_dim
where s_store_sk = ss_store_sk
and  ss_sold_date_sk = d_date_sk and d_year = 2001
and((ss_hdemo_sk=hd_demo_sk
  and cd_demo_sk = ss_cdemo_sk
  and cd_marital_status = 'M'
  and cd_education_status = 'Advanced Degree'
  and ss_sales_price between 100.00 and 150.00
  and hd_dep_count = 3
    )or
    (ss_hdemo_sk=hd_demo_sk
  and cd_demo_sk = ss_cdemo_sk
  and cd_marital_status = 'S'
  and cd_education_status = 'College'
  and ss_sales_price between 50.00 and 100.00
  and hd_dep_count = 1
    ) or
    (ss_hdemo_sk=hd_demo_sk
  and cd_demo_sk = ss_cdemo_sk
  and cd_marital_status = 'W'
  and cd_education_status = '2 yr Degree'
  and ss_sales_price between 150.00 and 200.00
  and hd_dep_count = 1
    ))
and((ss_addr_sk = ca_address_sk
  and ca_country = 'United States'
  and ca_state in ('TX', 'OH', 'TX')
  and ss_net_profit between 100 and 200
    ) or
    (ss_addr_sk = ca_address_sk
  and ca_country = 'United States'
  and ca_state in ('OR', 'NM', 'KY')
  and ss_net_profit between 150 and 300
```

```
    ) or
    (ss_addr_sk = ca_address_sk
  and ca_country = 'United States'
  and ca_state in ('VA', 'TX', 'MS')
  and ss_net_profit between 50 and 250
    ))
''',
'''-- start query 14 in stream 0 using template query14.tpl and seed
QUALIFICATION
with  cross_items as
(select i_item_sk ss_item_sk
from item,
(select iss.i_brand_id brand_id
    ,iss.i_class_id class_id
    ,iss.i_category_id category_id
from store_sales
    ,item iss
    ,date_dim d1
where ss_item_sk = iss.i_item_sk
  and ss_sold_date_sk = d1.d_date_sk
  and d1.d_year between 1999 AND 1999 + 2
intersect
select ics.i_brand_id
    ,ics.i_class_id
    ,ics.i_category_id
from catalog_sales
    ,item ics
    ,date_dim d2
where cs_item_sk = ics.i_item_sk
  and cs_sold_date_sk = d2.d_date_sk
  and d2.d_year between 1999 AND 1999 + 2
intersect
select iws.i_brand_id
    ,iws.i_class_id
    ,iws.i_category_id
from web_sales
    ,item iws
    ,date_dim d3
where ws_item_sk = iws.i_item_sk
  and ws_sold_date_sk = d3.d_date_sk
  and d3.d_year between 1999 AND 1999 + 2)
where i_brand_id = brand_id
      and i_class_id = class_id
      and i_category_id = category_id
),
avg_sales as
(select avg(quantity*list_price) average_sales
  from (select ss_quantity quantity
```

```
            ,ss_list_price list_price
      from store_sales
          ,date_dim
      where ss_sold_date_sk = d_date_sk
        and d_year between 1999 and 1999 + 2
      union all
      select cs_quantity quantity
            ,cs_list_price list_price
      from catalog_sales
          ,date_dim
      where cs_sold_date_sk = d_date_sk
        and d_year between 1999 and 1999 + 2
      union all
      select ws_quantity quantity
            ,ws_list_price list_price
      from web_sales
          ,date_dim
      where ws_sold_date_sk = d_date_sk
        and d_year between 1999 and 1999 + 2) x)
  select  channel, i_brand_id,i_class_id,i_category_id,sum(sales),
sum(number_sales)
from(
      select 'store' channel, i_brand_id,i_class_id
            ,i_category_id,sum(ss_quantity*ss_list_price) sales
            , count(*) number_sales
      from store_sales
          ,item
          ,date_dim
      where ss_item_sk in (select ss_item_sk from cross_items)
        and ss_item_sk = i_item_sk
        and ss_sold_date_sk = d_date_sk
        and d_year = 1999+2
        and d_moy = 11
      group by i_brand_id,i_class_id,i_category_id
      having sum(ss_quantity*ss_list_price) > (select average_sales from
avg_sales)
      union all
      select 'catalog' channel, i_brand_id,i_class_id,i_category_id,
sum(cs_quantity*cs_list_price) sales, count(*) number_sales
      from catalog_sales
          ,item
          ,date_dim
      where cs_item_sk in (select ss_item_sk from cross_items)
        and cs_item_sk = i_item_sk
        and cs_sold_date_sk = d_date_sk
        and d_year = 1999+2
        and d_moy = 11
      group by i_brand_id,i_class_id,i_category_id
```

```
      having sum(cs_quantity*cs_list_price) > (select average_sales from
avg_sales)
      union all
      select 'web' channel, i_brand_id,i_class_id,i_category_id,
sum(ws_quantity*ws_list_price) sales , count(*) number_sales
      from web_sales
          ,item
          ,date_dim
      where ws_item_sk in (select ss_item_sk from cross_items)
        and ws_item_sk = i_item_sk
        and ws_sold_date_sk = d_date_sk
        and d_year = 1999+2
        and d_moy = 11
      group by i_brand_id,i_class_id,i_category_id
      having sum(ws_quantity*ws_list_price) > (select average_sales from
avg_sales)
) y
group by rollup (channel, i_brand_id,i_class_id,i_category_id)
order by channel,i_brand_id,i_class_id,i_category_id
  limit 100;''',

'''select  ca_zip
      ,sum(cs_sales_price)
from catalog_sales
    ,customer
    ,customer_address
    ,date_dim
where cs_bill_customer_sk = c_customer_sk
  and c_current_addr_sk = ca_address_sk
  and ( substr(ca_zip,1,5) in ('85669', '86197','88274','83405','86475',
                               '85392', '85460', '80348', '81792')
        or ca_state in ('CA','WA','GA')
        or cs_sales_price > 500)
  and cs_sold_date_sk = d_date_sk
  and d_qoy = 2 and d_year = 2001
group by ca_zip
order by ca_zip
  limit 100'''
]
```

Create_Engine() and retrieve snowflake credentials from configuration file – define tables:

Define two Snowflake database connections using the create_engine function from the SQLAlchemy library. These connections are used to query data from two different schemas within the same Snowflake account.

tables: This is a list of strings containing the names of the tables (query1, query2, query3, query4, and query5) that will be used later in the code.

These two connections, engine_old and engine_new, can be used to execute SQL queries on the respective databases and schemas.

For improved readability and maintainability, store sensitive information like user, password, and account details in environment variables or a configuration file. Here's the updated code:

```
import os
from sqlalchemy import create_engine
from snowflake.sqlalchemy import URL

tables = ['query1', 'query2', 'query3', 'query4', 'query5']

engine_old = create_engine(URL(
    account = os.environ['SNOWFLAKE_ACCOUNT'],
    user = os.environ['SNOWFLAKE_USER'],
    password = os.environ['SNOWFLAKE_PASSWORD'],
    database = 'SNOWFLAKE_SAMPLE_DATA',
    schema = 'TPCDS_SF10TCL',
    warehouse = 'COMPUTE_WH',
    role = 'ACCOUNTADMIN',
))

engine_new = create_engine(URL(
    account = os.environ['SNOWFLAKE_ACCOUNT'],
    user = os.environ['SNOWFLAKE_USER'],
    password = os.environ['SNOWFLAKE_PASSWORD'],
    database = 'midterm',
```

```
    schema = 'public',
    warehouse = 'COMPUTE_WH',
    role = 'ACCOUNTADMIN',
))
```

```
tables=['query1','query2','query3','query4','query5']

engine_old= create_engine(URL(
    account = 'pinvdzu-ljb05593',
    user = 'gamer9797123',
    password = 'Gamer9797123)',
    database = 'SNOWFLAKE_SAMPLE_DATA',
    schema = 'TPCDS_SF10TCL',
    warehouse = 'COMPUTE_WH',
    role='ACCOUNTADMIN',
))
engine_new = create_engine(URL(
    account = 'pinvdzu-ljb05593',
    user = 'gamer9797123',
    password = 'Gamer9797123)',
    database = 'midterm',
    schema = 'public',
    warehouse = 'COMPUTE_WH',
    role='ACCOUNTADMIN',
))
```

CLV Model AND Streamlit Interface

```
# Iterate through the list of queries and corresponding table names
for query, table_name in zip(queries, tables):
```

```
    # Connect to the old database engine

    connection = engine_old.connect()
    print(table_name, "old Connected")

    # Read data from the old database using the query

    res = pd.read_sql(query, connection)

    # Dispose the old database connection

    engine_old.dispose()

    # Connect to the new database engine

    connection = engine_new.connect()

    # Write the data to the new database table

    res.to_sql(table_name, connection, index=False, if_exists='replace')
    print(table_name, "new  Connected")

    # Dispose the new database connection


    engine_new.dispose()

# Display the code block in the Streamlit app

st.code(p1code, language='python')

# In the second column, iterate through the list of questions, table names, and tabs

for table_name, tab, question in zip(tables, tab_names, questions):
    with tab:
        # Create a SQL query to fetch data from the table

        query = "select * from " + str(table_name)

        # Connect to the part1_engine database

        connection = part1_engine.connect()

        # Read data from the table using the query

        res = pd.read_sql(query, connection)

        # Display the query number and question in the Streamlit app

        title = "Query number: " + str(table_name)
        st.header(title)
        st.code(question)
```

```
        # Display the result in the Streamlit app

        st.write(res)

    # Dispose the part1_engine database connection

    part1_engine.dispose()

# Function to run the queries for the Streamlit app part 2

def run_query(dob_list, education_option, gender_option, dept_option, credit_option, marital_option):
    # Define the SQL queries with the selected options
    # Replace this comment with the actual SQL query definitions
    return query, query2

# Main function for the Streamlit app

def main():
    # Define the part2_engine database connection
    part2_engine = create_engine(URL(
        account = 'pinvdzu-ljb05593',
        user = 'gamer9797123',
        password = 'Gamer9797123)',
        database = 'TPCDS_XGBOOST',
        schema = 'DEMO',
        warehouse = 'COMPUTE_WH',
        role='ACCOUNTADMIN',
    ))
    # Connect to the part2_engine database

    connection = part2_engine.connect()

    # Use columns and widgets to create the Streamlit app layout

    col1, col2 = st.columns(2)
    with col1:
        # Add Streamlit widgets for user inputs, like sliders and multiselects
        # Replace the following comments with actual Streamlit widgets

        # dob_slider = ...
        # education_option = ...
        # gender_option = ...
        # dept_option = ...
        # credit_option = ...
        # marital_option = ...

        # Call the run_query function to get the SQL queries

        query, query2 = run_query(dob_list, education_option, gender_option, dept_option, credit_option,
marital_option)

        # Fetch data from the database using the queries and display the results in the app
```

```
        res2 = pd.read_sql(query, connection)

       # Fetch data from the database using the queries and display the results in the app
# Execute the SQL query by passing it to the `read_sql` function along with the database connection

res2 = pd.read_sql(query, connection)

# The `res2` variable now contains a DataFrame with the results of the executed SQL query.
# We assume that the first row (0) and the first column (0) of the DataFrame contain the predicted sales value.
# Similarly, we assume that the first row (0) and the second column (1) of the DataFrame contain the actual sales value.

# To display the predicted sales value, we access the value using the `loc` function of the DataFrame:
# res2.loc[0][0] - This accesses the value in the first row and the first column of the DataFrame.
# We then divide the value by 1,000,000,000 (1 billion) to convert it into billions.
# Finally, we round the value to 2 decimal places using the `round` function and display it in the Streamlit app.


st.write('Predicted Sales:', round(res2.loc[0][0] / 1000000000, 2), 'billion USD')

# Similarly, to display the actual sales value, we access the value using the `loc` function of the DataFrame:
# res2.loc[0][1] - This accesses the value in the first row and the second column of the DataFrame.
# We then divide the value by 1,000,000,000 (1 billion) to convert it into billions.
# Finally, we round the value to 2 decimal places using the `round` function and display it in the Streamlit app.


st.write('Actual Sales:', round(res2.loc[0][1] / 1000000000, 2), 'billion USD')
```

Customer_lifetime_value.py -- running this will run:
https://ne-tpcds-customer-lifetime-valuecustomer-lifetime-value-au9kfr.streamlit.app/

**Part 1 of the application provides an interface to execute five assigned TPC-DS queries using Snowflake SQLAlchemy in Python. Users can input the required variables following the Qualification Substitution Parameters in the TPC-DS documentation. The application allows users to validate their input values for specific constraints, such as date fields. The interface displays the results of the executed queries.

Part 2 of the application allows users to input customer features and obtain their customer lifetime value (CLV) prediction. The application employs a trained CLV model using the TPC-DS dataset. The feature engineering and model training are executed through the xgboost_tpcds.ipynb Jupyter notebook, where the model is trained and hosted on a platform. The **Customer_lifetime_value.py file is the main Streamlit application that interacts with** the hosted model, allowing users to input customer features and view their CLV

prediction.

To use the CLV application, users can input customer features, such as the birth year, gender, marital status, credit rating, education status, and dependents count. The application uses these features as inputs to the trained XGBoost model to calculate the customer's expected lifetime value. The interface displays the predicted CLV value, allowing users to understand the potential value of each customer to their business. Overall, the web application provides a user-friendly interface for both querying the TPC-DS dataset and predicting customer lifetime value. The application is built using Snowflake SQLAlchemy in Python, and it provides a great example of how to build and deploy machine learning models with Snowflake's data platform.

Part-2 – Training and modeling an XGBoost model for CLV prediction (xgboost_tpcds.ipynb)

Calculating Customer Lifetime Value (CLV) from provided TPC-DS dataset

The objective here is to build a model for calculating Customer Lifetime Value (CLV) using the provided TPC-DS dataset. CLV is a key performance indicator for any business that is looking to grow and retain its customer base. The CLV model aims to predict the total value that a customer will generate for a company over the lifetime of their relationship.

To achieve this goal, the first step is to perform feature engineering and data preprocessing on the TPC-DS dataset. This includes aggregating sales data across all channels (web, store, catalogue) and joining it to customer demographic data. The resulting dataset is then split into training and testing sets, and a machine learning model is trained on the prepared data.

In this project, an XGBoost model is used for training, which is a popular gradient boosting framework that has been shown to perform well on many different types of datasets. Once the model is trained, it is saved and hosted on a cloud platform to be accessed by the Streamlit interface.

The Streamlit interface developed in this project allows users to input the necessary features for CLV calculation, such as birth year, gender, marital status, credit rating, education status, and zip code. Upon invoking the model, the interface displays the predicted CLV for the given customer.

Getting set up

Import necessary libraries and sets up a connection to a Snowflake database using Snowpark, which is a DataFrame API provided by Snowflake for data processing and transformation.

Import libraries:
The code imports the required libraries and modules, including snowflake.snowpark, functions as F, Session, and version as v. It also imports the json library to handle JSON data.

Load connection details from a JSON file:
The code reads a JSON file named connection.json to obtain the Snowflake connection details, such as the username, password, account, and warehouse.

Set up connection parameters:
The CONNECTION_PARAMETERS dictionary is created using the details loaded from the JSON file.

Create a Snowpark session:
The Session object is created by calling the Session.builder.configs() method and passing the CONNECTION_PARAMETERS dictionary. The create() method is then called to establish the connection to the Snowflake database.

```
// importing libraries and setting up snowflake

import snowflake.snowpark
from snowflake.snowpark import functions as F
from snowflake.snowpark.session import Session
from snowflake.snowpark import version as v
import json

with open('connection.json') as f:
    data = json.load(f)
    USERNAME = data['user']
    PASSWORD = data['password']
```

```
    SF_ACCOUNT = data['account']
    SF_WH = data['warehouse']

CONNECTION_PARAMETERS = {
   "account": SF_ACCOUNT,
   "user": USERNAME,
   "password": PASSWORD,
}

session = Session.builder.configs(CONNECTION_PARAMETERS).create()
```

Create Databases, warehouses, schemas and configure them

Set up the Snowflake environment by creating necessary databases, schemas, and warehouses, and configuring warehouse settings to ensure the proper resources are allocated for the tasks at hand.

```
# Create the 'snowflake_sample_data' database if it doesn't exist, using the shared 'sample_data' from 'sfc_samples'

session.sql('''create database if not exists snowflake_sample_data from share sfc_samples.sample_data''').collect()

# Create the 'tpcds_xgboost' database if it doesn't exist
session.sql('CREATE DATABASE IF NOT EXISTS tpcds_xgboost').collect()

# Create the 'demo' schema within the 'tpcds_xgboost' database if it doesn't exist

session.sql('CREATE SCHEMA IF NOT EXISTS tpcds_xgboost.demo').collect()

# Create or replace a warehouse named 'FE_AND_INFERENCE_WH' with a warehouse size of '3X-LARGE'

session.sql("create or replace warehouse FE_AND_INFERENCE_WH with warehouse_size='3X-LARGE'").collect()

# Create or replace a warehouse named 'snowpark_opt_wh' with a warehouse size of 'MEDIUM' and type 'SNOWPARK-OPTIMIZED'

session.sql("create or replace warehouse snowpark_opt_wh with warehouse_size = 'MEDIUM' warehouse_type = 'SNOWPARK-OPTIMIZED'").collect()
```

```
# Set the maximum concurrency level of the 'snowpark_opt_wh' warehouse to 1

session.sql("alter warehouse snowpark_opt_wh set max_concurrency_level = 1").collect()

# Use the 'FE_AND_INFERENCE_WH' warehouse for the session

session.use_warehouse('FE_AND_INFERENCE_WH')

# Instructions for selecting the TPC-DS Dataset size (100 or 10) to use and the recommended
warehouse size
```

****See the link provided for more information on the dataset size

Set the TPC-DS dataset size, defines the Snowflake Sample Database name, and determines the schema based on the dataset size. Then, it accesses the relevant tables in the sample database using the determined schema.

```
# Set the TPC-DS dataset size parameter (choose either 100 or 10)
TPCDS_SIZE_PARAM = 100
# Define the name of the Snowflake Sample Database (might be different depending on your setup)
SNOWFLAKE_SAMPLE_DB = 'SNOWFLAKE_SAMPLE_DATA'

# Determine the TPCDS schema based on the chosen dataset size

if TPCDS_SIZE_PARAM == 100:
    TPCDS_SCHEMA = 'TPCDS_SF100TCL'
elif TPCDS_SIZE_PARAM == 10:
    TPCDS_SCHEMA = 'TPCDS_SF10TCL'
else:
    # Raise an error if an invalid dataset size is selected
    raise ValueError("Invalid TPCDS_SIZE_PARAM selection")

# Access the relevant tables in the sample database using the determined schema

store_sales = session.table(f'{SNOWFLAKE_SAMPLE_DB}.{TPCDS_SCHEMA}.store_sales')

catalog_sales = session.table(f'{SNOWFLAKE_SAMPLE_DB}.{TPCDS_SCHEMA}.catalog_sales')

web_sales = session.table(f'{SNOWFLAKE_SAMPLE_DB}.{TPCDS_SCHEMA}.web_sales')
```

```
date = session.table(f'{SNOWFLAKE_SAMPLE_DB}.{TPCDS_SCHEMA}.date_dim')

dim_stores = session.table(f'{SNOWFLAKE_SAMPLE_DB}.{TPCDS_SCHEMA}.store')

customer = session.table(f'{SNOWFLAKE_SAMPLE_DB}.{TPCDS_SCHEMA}.customer')

address = session.table(f'{SNOWFLAKE_SAMPLE_DB}.{TPCDS_SCHEMA}.customer_address')

demo = session.table(f'{SNOWFLAKE_SAMPLE_DB}.{TPCDS_SCHEMA}.customer_demographics')
```

Feature Engineering

Aggregate sales data across all channels (web, store, and catalog) and joining the resulting data with customer demographic data. It calculates total sales for each customer and then unions the results for all channels. Finally, it joins the aggregated data with the customer, address, and customer_demographics tables to create a complete dataset for further analysis.

```
# Set the TPC-DS dataset size parameter (choose either 100 or 10)

TPCDS_SIZE_PARAM = 100

# Define the name of the Snowflake Sample Database (might be different depending on your setup)

# Aggregate sales by customer across all channels (web, store, catalog) and join that to customer demographic data

# Group by customer ID and calculate total sales for store_sales

store_sales_agged =
store_sales.group_by('ss_customer_sk').agg(F.sum('ss_sales_price').as_('total_sales'))

# Group by customer ID and calculate total sales for web_sales

web_sales_agged =
web_sales.group_by('ws_bill_customer_sk').agg(F.sum('ws_sales_price').as_('total_sales'))

# Group by customer ID and calculate total sales for catalog_sales

catalog_sales_agged =
catalog_sales.group_by('cs_bill_customer_sk').agg(F.sum('cs_sales_price').as_('total_sales'))
```

```
# Rename columns to match customer_sk across all tables

store_sales_agged = store_sales_agged.rename('ss_customer_sk', 'customer_sk')

web_sales_agged = web_sales_agged.rename('ws_bill_customer_sk', 'customer_sk')

catalog_sales_agged = catalog_sales_agged.rename('cs_bill_customer_sk', 'customer_sk')

# Union store_sales_agged and web_sales_agged
total_sales = store_sales_agged.union_all(web_sales_agged)

# Union the result with catalog_sales_agged
total_sales = total_sales.union_all(catalog_sales_agged)

# Group by customer_sk and calculate the total_sales

total_sales = total_sales.group_by('customer_sk').agg(F.sum('total_sales').as_('total_sales'))

# Select relevant columns from the customer table

customer = customer.select('c_customer_sk', 'c_current_hdemo_sk', 'c_current_addr_sk',
'c_customer_id', 'c_birth_year')

# Join the customer table with the address table

customer = customer.join(address.select('ca_address_sk', 'ca_zip'), customer['c_current_addr_sk'] ==
address['ca_address_sk'])

# Join the customer table with the customer_demographics table

customer = customer.join(demo.select('cd_demo_sk', 'cd_gender', 'cd_marital_status',
'cd_credit_rating', 'cd_education_status', 'cd_dep_count'),
                         customer['c_current_hdemo_sk'] == demo['cd_demo_sk'])

# Rename 'c_customer_sk' column to 'customer_sk'

customer = customer.rename('c_customer_sk', 'customer_sk')
customer.show()
```

Congratulations

Congratulations, we now have a Streamlit interface that connects to the Snowflake database using sqlalchemy in Python and executes the TPC-DS queries. The interface allows users to input Qualification Substitution Parameters as variables and validates user inputs to ensure they are within the allowed range of values. We also have a model that estimates the lifetime value of each customer and segments them based on their value to the business. We can use this information to create personalized marketing campaigns to retain high-value customers. We can host the model using cloud services and create a Streamlit interface that allows users to input customer data and receive personalized marketing recommendations.